OPERATING SYSTEMS

ASSIGNMENT 2 – CO2

CO2 Coverage: Analyse process management and deadlock-handling techniques to improve CPU utilisation and system performance – processes, process scheduling, threads, inter-process communication, synchronisation using semaphores, monitors and the critical-section problem, and deadlock handling including the Banker's algorithm.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment2** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Q1 – Processes and Process States 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Online Examination Server

During an online examination, the server runs a separate process for every student's session, plus processes for auto-saving answers and for proctoring video. When 2,000 students log in together, the server becomes slow and some sessions appear to "hang".

Your Challenge

Draw the process-state diagram (new, ready, running, waiting, terminated) and the process control block (PCB). Write a program that creates child processes with fork() and observe them with ps and /proc.

Evaluation

Evaluate where the time goes when 2,000 processes are active: context-switch cost, memory per process, and time spent waiting for I/O. Judge whether a separate process per student is a sound design.

Design Decision

Decide between one process per student, one thread per student and a fixed pool of worker processes, and defend your decision.

Alternative Concept and Complexity Analysis

If each student session were a thread inside one server process, instead of a separate process, what would the solution be? Evaluate the context-switch time and the memory per session (a few KB of stack versus a full address space), and the effect of one session crashing.

Evidence Expected

A state diagram, a PCB description, a working fork() program with ps evidence and a defended design decision.

Q2 – CPU Scheduling 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Cloud Video-Encoding Service

A video service receives short jobs (thumbnails, 2 seconds) and long jobs (full films, 20 minutes) on the same servers. Customers complain that small jobs wait behind large ones, while the company wants the processors fully used.

Your Challenge

Compute the Gantt chart, waiting time, turnaround time and response time for the same set of jobs under FCFS, SJF, SRTF, priority and Round Robin scheduling. Implement Round Robin.

Evaluation

Evaluate how the time quantum affects Round Robin (try 1, 3, 5 and 20), the risk of starvation in SJF and priority scheduling, and how ageing removes it. Explain how a multilevel feedback queue treats short and long jobs.

Design Decision

Recommend the scheduling policy for the video service and justify it with your calculated averages.

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>

#define N 4

int main(void)
{
    const char *name[N] = {"P1", "P2", "P3", "P4"};
    int arrival[N] = {0, 1, 2, 3};
    int burst[N] = {8, 4, 9, 5};
    int quantum = 3;

    int left[N], finish[N], queue[64], head = 0, tail = 0, added[N] = {0};
    for (int i = 0; i < N; i++) left[i] = burst[i];
    int time = 0, done = 0;
    queue[tail++] = 0;
    added[0] = 1;
    printf("Gantt:");
    while (done < N) {
        int p = queue[head++];
        // TODO 1: Run p for min(quantum, left[p]) time units: print " P(time-newtime)",
        //   advance time, reduce left[p]; add every process that has now arrived and is
        //   not yet added to the queue; then put p back at the end of the queue if it
        //   still has time left, otherwise record finish[p] and count it as done

    }
    double tw = 0, tt = 0;
    printf("\nProcess  TAT  WT\n");
    for (int i = 0; i < N; i++) {
        int tat = finish[i] - arrival[i], wt = tat - burst[i];
        tt += tat;
        tw += wt;
        printf("%-7s %4d %3d\n", name[i], tat, wt);
    }
    printf("Average TAT = %.2f, average WT = %.2f\n", tt / N, tw / N);
    return 0;
}
```

Expected output:

```
Gantt: P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21)
P1(21-23) P3(23-26)
Process  TAT  WT
P1        23  15
P2        15  11
P3        24  15
P4        18  13
Average TAT = 20.00, average WT = 13.50
```

Run the program with quantum 1 and quantum 20, and evaluate how the average waiting time and the number of context switches change.

Alternative Concept and Complexity Analysis

If the service used a multilevel feedback queue (short quantum for new jobs, longer quanta for jobs that use them up) instead of a single Round Robin queue, what would the solution be? Evaluate the average waiting time for short and long jobs, the scheduling cost per decision (O(1) per queue versus O(n) for SJF) and the risk of starvation.

Evidence Expected

Correct Gantt charts and averages for every algorithm, a working Round Robin program, a quantum study and a justified recommendation.

Q3 – Threads — 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Traffic-Camera Number-Plate Reader

A city traffic system reads number plates from 64 cameras. The first version processes the cameras one after another and falls behind during peak hours.

Your Challenge

Process the camera frames with POSIX threads (one thread per camera, and a pool of 4, 8 and 16 threads). Determine:

- Which resources threads share and which each thread has of its own
- The difference between user-level and kernel-level threads and the many-to-one, one-to-one and many-to-many models
- The speed-up obtained on your machine, and why it stops increasing
- What happens to shared counters when two threads update them without protection

Alternative Concept and Complexity Analysis

If the camera frames were processed by several processes instead of threads, what would the solution be? Evaluate the creation and switching time, the memory used (separate address spaces versus shared), and how the results would be combined (IPC versus shared variables).

Evidence Expected

A working multithreaded program, measured speed-ups for each pool size and a reasoned evaluation of the thread models.

Q4 – Inter-Process Communication — 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Weather Station Data Pipeline

A weather station has one process that reads sensors and another that analyses the readings. They must exchange data continuously without losing readings.

Your Challenge

Implement the exchange using a pipe, and then using shared memory or a message queue. Determine:

- How the pipe signals end of data to the reader
- The difference between the shared-memory and message-passing models
- Which method copies the data and which does not
- Which method needs extra synchronisation, and why

Growth Investigation

Measure the time to transfer 1 MB, 10 MB and 100 MB with each method, and evaluate the results.

Complete the Code

Complete TODO 1 below and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

/* The parent (sensor reader) sends readings to the child (analyser) through a pipe. */
int main(void)
{
    int fd[2];
    if (pipe(fd) == -1) { perror("pipe"); return 1; }
    pid_t pid = fork();
    if (pid == 0) {
        close(fd[1]);                          /* child only reads */
        int r, count = 0, sum = 0, max = 0;
        // TODO 1: Read one int at a time from fd[0] until read returns 0 (the parent
        //   closed the pipe); keep the count, the sum and the maximum

        printf("analyser: %d readings, average %.1f, max %d\n", count,
               count ? (double)sum / count : 0.0, max);
        close(fd[0]);
        fflush(stdout);
        _exit(0);
    }
    close(fd[0]);                              /* parent only writes */
    int readings[] = {31, 33, 36, 34, 41, 38};
    for (int i = 0; i < 6; i++)
        if (write(fd[1], &readings[i], sizeof readings[i]) < 0) perror("write");
    close(fd[1]);                              /* signals end of data to the child */
    waitpid(pid, NULL, 0);
    printf("reader: done\n");
    return 0;
}
```

Expected output:

```
analyser: 6 readings, average 35.5, max 41
reader: done
```

Remove close(fd[1]) in the parent and explain why the child never finishes.

Alternative Concept and Complexity Analysis

If the readings were exchanged through shared memory with a semaphore instead of a pipe, what would the solution be? Evaluate the time to transfer n readings (two copies through the kernel for a pipe versus none for shared memory) and the synchronisation needed.

Evidence Expected

Two working IPC implementations, measured transfer times and a justified choice of method.

Q5 – Synchronisation: Critical Sections, Semaphores and Monitors — 11 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Railway Ticket Booking

Thousands of agents book seats on the same trains at the same time. Without synchronisation the same seat is sometimes sold twice; with a single global lock the system becomes very slow.

Design Starting Point

You are given only the following principle:

A solution to the critical-section problem must guarantee mutual exclusion, progress and bounded waiting.

Your Challenge

Convert this principle into a design. Determine:

- Why Peterson's solution satisfies the three requirements for two processes, and why it may fail on modern processors
- How counting and binary semaphores (wait and signal) protect the seat counter
- How the bounded-buffer (producer–consumer), readers–writers and dining-philosophers problems are solved with semaphores
- How a monitor with condition variables makes the solution easier to get right

Test Your Design

Run the bounded-buffer program with 1 and with 4 producers and consumers, and show that every item is consumed exactly once.

Deployment Decision

Decide between one lock for the whole booking system, one lock per train and one lock per coach. Measure the throughput of each and defend your choice.

Complete the Code

Complete TODO 1 and TODO 2 below using the semaphores and the mutex, and execute the program (compile with -pthread).

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 5
#define ITEMS 20

int buffer[SIZE], in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
long consumed_sum = 0;

void *producer(void *arg)
{
    (void)arg;
    for (int item = 1; item <= ITEMS; item++) {
        // TODO 1: Wait on empty, lock the mutex, put item at buffer[in], advance in
        //   circularly, unlock, then post full

    }
    return NULL;
}

void *consumer(void *arg)
{
    (void)arg;
    for (int k = 0; k < ITEMS; k++) {
        // TODO 2: Wait on full, lock the mutex, take the item at buffer[out], advance
        //   out circularly, unlock, post empty and add the item to consumed_sum

    }
    return NULL;
}

int main(void)
{
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_t p, c;
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    printf("items consumed: %d, sum = %ld (expected %d)\n", ITEMS, consumed_sum,
           ITEMS * (ITEMS + 1) / 2);
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

Expected output:

```
items consumed: 20, sum = 210 (expected 210)
```

Run the program 100 times and show that the sum is always 210. Then remove the mutex and evaluate whether the result is still correct with several producers and consumers.

Alternative Concept and Complexity Analysis

If the shared counter were protected by atomic operations (C11 atomic_fetch_add) or by a spin lock instead of a semaphore and mutex, what would the solution be? Evaluate the time per update with 1, 4 and 16 threads, the CPU time wasted by spinning, and which method suits a long critical section.

Evidence Expected

A student-derived synchronisation design, working semaphore code, test evidence and a measured, defended locking decision.

DAY-4 DESIGN CHALLENGE

Q6 – Deadlock Handling and the Banker's Algorithm — 13 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Shared Laboratory Equipment Scheduler

A research centre runs jobs that need several shared resources at once (GPUs, licences for simulation software and high-speed storage). Jobs sometimes wait for each other forever and the whole cluster stops until an administrator intervenes.

A. Characterise Deadlock – 2 Marks

State the four necessary conditions for deadlock and show them in a resource-allocation graph for the cluster.

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>
#include <string.h>

#define P 5
#define R 3

int avail[R] = {3, 3, 2};
int maxm[P][R] = {{7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}};
int alloc[P][R] = {{0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}};

/* Returns 1 if the state is safe and stores a safe sequence in seq. */
int is_safe(int seq[P])
{
    int work[R], finish[P] = {0}, count = 0;
    memcpy(work, avail, sizeof work);
    while (count < P) {
        int found = 0;
        for (int p = 0; p < P; p++) {
            if (finish[p]) continue;
            // TODO 1: If need = maxm[p] - alloc[p] is <= work for every resource, let p
            //   finish: add alloc[p] to work, mark finish[p], append p to seq and set
            //   found = 1

        }
        if (!found) return 0;
    }
    return 1;
}

void request(int p, const int req[R])
{
    for (int r = 0; r < R; r++)
        if (req[r] > maxm[p][r] - alloc[p][r] || req[r] > avail[r]) {
            printf("P%d request: must wait (not available)\n", p);
            return;
        }
    for (int r = 0; r < R; r++) { avail[r] -= req[r]; alloc[p][r] += req[r]; }
    int seq[P];
    if (is_safe(seq)) {
        printf("P%d request: granted, safe sequence", p);
        for (int i = 0; i < P; i++) printf(" P%d", seq[i]);
        printf("\n");
    } else {
        for (int r = 0; r < R; r++) { avail[r] += req[r]; alloc[p][r] -= req[r]; }
        printf("P%d request: denied (unsafe)\n", p);
    }
}

int main(void)
{
    int seq[P];
    if (is_safe(seq)) {
        printf("initial state safe:");
        for (int i = 0; i < P; i++) printf(" P%d", seq[i]);
        printf("\n");
    }
    request(1, (int[R]){1, 0, 2});
    request(4, (int[R]){3, 3, 0});
    request(0, (int[R]){0, 2, 0});
    return 0;
}
```

Expected output:

```
initial state safe: P1 P3 P4 P0 P2
P1 request: granted, safe sequence P1 P3 P4 P0 P2
P4 request: must wait (not available)
P0 request: denied (unsafe)
```

Show by hand why the request of P0 for (0, 2, 0) leads to an unsafe state.

Alternative Concept and Complexity Analysis

If deadlock were handled by detection and recovery (running a detection algorithm every few minutes and aborting a job) instead of the Banker's algorithm, what would the solution be? Evaluate the time per request ($O(m \times n^2)$ for every Banker's check versus no check), the cost of detection, and the work lost when a job is aborted.

Evidence Expected

A correct resource-allocation graph identifying all four conditions.

B. Compare the Strategies – 3 Marks

Evaluate prevention (removing one condition), avoidance (Banker's algorithm), detection and recovery, and ignoring deadlock (the ostrich approach) for the cluster.

Evidence Expected

A reasoned comparison of the four strategies for this environment.

C. Apply the Banker's Algorithm – 4 Marks

Implement the safety algorithm and the resource-request algorithm. Find a safe sequence for the given state and decide whether each request can be granted.

Evidence Expected

A working implementation with correct safe sequences and decisions.

D. Predict Before Implementing – 1 Mark

Predict how the running time of the Banker's algorithm grows with the number of processes n and resource types m.

Evidence Expected

A clear, testable prediction.

E. Measure – 1 Mark

Measure the time of the safety algorithm for 10, 100 and 1,000 processes and compare it with your prediction.

Evidence Expected

Measured times compared with the prediction.

F. Defend the Final Design – 2 Marks

Recommend the deadlock strategy for the research centre and justify it, including the cost of the Banker's algorithm and the need to know maximum demands in advance.

Evidence Expected

A critical, evidence-based recommendation.

MARK DISTRIBUTION

Question	Assessed Area	Bloom's Level	Marks
Q1	Processes and process states	K5 – Evaluate	6
Q2	CPU scheduling	K5 – Evaluate	8
Q3	Threads	K5 – Evaluate	6

Question	Assessed Area	Bloom's Level	Marks
Q4	Inter-process communication	K5 – Evaluate	6
Q5	Synchronisation: critical sections, semaphores and monitors	K5 – Evaluate	11
Q6	Day-4 Design Challenge – Deadlock handling and Banker's algorithm	K5 – Evaluate	13
	TOTAL		**50**

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.

- Your assignment report containing the required evidence for Q1–Q6
- C source code for all programs
- Gantt charts, diagrams, outputs and timings used as evidence

Assessment Verification

Students may be asked to explain any part of their design, code, calculations or results during a brief verification discussion/viva.

Overall Assessment Expectation

> **Model → Design → Implement → Measure → Compare → Evaluate → Recommend**

ASSIGNMENT 2 (CO2) — ANSWER KEY / SOLUTION NOTES

These notes answer every question of Assignment 2 (process management, scheduling, threads, IPC, synchronisation and deadlock handling). They are written as a model solution to guide the student's own report, code execution, GitHub submission and viva preparation. Each student must still execute every program on their own Linux machine, capture real screenshots, ps/strace/time output and write the README.md as instructed on the cover pages.

Q1 – Processes and Process States (6 Marks)

Process-state diagram: the five standard states are New → Ready → Running → (Waiting/Blocked) → Terminated. A process is created (New), is admitted to the ready queue (Ready), is dispatched by the scheduler onto the CPU (Running), may be pre-empted back to Ready or may request I/O and move to Waiting until the I/O completes (back to Ready), and finally exits (Terminated), after which its exit status remains until the parent reaps it (zombie, a sub-state of Terminated).

PCB (Process Control Block) contents: process ID and parent PID; process state; saved CPU register values (including the program counter and stack pointer) for context-switch restore; scheduling information (priority, CPU-usage accounting); memory-management information (page-table pointer/base, segment limits); accounting information (CPU time used, time limits); I/O status information (open file descriptors, pending I/O requests, allocated devices).

Observing child processes: a fork()-based test program creates N child processes, each of which can be observed with ps -ef --forest (shows the parent-child tree) and with the per-process files under /proc/<pid>/status (State:, VmRSS: for memory) and /proc/<pid>/stat (fields including state, utime, stime).

Evaluation — 2,000 active processes: each process needs its own PCB plus page tables (memory cost scales roughly O(number of processes × page-table size), even though data pages themselves may be shared/COW); each context switch between processes costs CPU time for saving/restoring registers and, because each has a separate address space, typically a TLB flush/reload, which is markedly more expensive than switching between threads of the same process; with 2,000 mostly I/O-waiting sessions (students reading a question, occasionally submitting an answer), most process time is actually spent in the Waiting state for network I/O, not Running — so the bottleneck is less raw CPU scheduling and more the fixed per-process memory and context-switch overhead multiplied by 2,000. A separate OS process per student is not a sound design at this scale: it works correctly but wastes memory and context-switch time on mostly-idle sessions.

Design decision: a fixed pool of worker processes (or threads) that each multiplex many idle student connections (an event-driven or thread-pool server model) is the soundest choice — it bounds the number of OS-level scheduling entities to a small, fixed number regardless of how many students are logged in, avoiding the O(2,000) memory/context-switch overhead of one-process-per-student while still giving a reasonable degree of isolation; one-thread-per-student is a middle ground (cheaper than one-process-per- student, since threads share one address space, but still creates one kernel scheduling entity per student and so still scales linearly with student count, unlike a bounded pool).

Alternative Concept and Complexity Analysis — one thread per session instead of one process per session: context-switch time between threads of the same process is lower than between processes, because there is no address-space/page-table switch (and often no TLB flush) — only the CPU registers and stack pointer need saving/restoring. Memory per session drops from a full address space (page tables plus any private, non-shared pages) to just a thread stack, commonly a few hundred KB to a few MB (often tunable down further), a large constant-factor saving when multiplied by thousands of sessions. The key risk is isolation: if one session's thread crashes (e.g., a null-pointer bug) or corrupts shared memory, it can take down the entire server process and every other student's session with it, whereas a crashing process only ends that one student's session — this is the direct trade-off threads make against processes for this workload.

Q2 – CPU Scheduling (8 Marks)

Worked Gantt charts for a representative job set (arrival/burst as commonly used: P1(0,8), P2(1,4), P3(2,9), P4(3,5)):

FCFS (run in arrival order): P1(0-8) P2(8-12) P3(12-21) P4(21-26). TAT: P1=8,P2=11,P3=19,P4=23 → avg TAT=15.25; WT: P1=0,P2=7,P3=10,P4=18 → avg WT=8.75. This matches the Task 2 Round-Robin program's own job set/results given in the question for the Round Robin case (quantum 3), reproduced by the completed program below.

SJF (non-preemptive, shortest burst first when CPU is free): orders by burst length among ready jobs at each decision point; typically reduces average waiting time versus FCFS for this mixed short/long job set because short jobs (like the 2-second thumbnail jobs) are not stuck behind a long job that arrived first.

SRTF (preemptive SJF): can pre-empt a running long job the instant a shorter job arrives, giving the lowest possible average waiting time of any non-priority-based algorithm for a given job set, at the cost of more context switches.

Priority scheduling: orders by assigned priority; risks starvation of low-priority (e.g., long film-encode) jobs if high-priority short jobs keep arriving — solved by ageing (gradually raising the priority of a job the longer it waits, guaranteeing it is eventually scheduled).

Round Robin (quantum q): cycles through the ready queue, running each job for at most q time units before pre-empting it to the back of the queue — gives every job (short or long) a bounded maximum initial response time, directly addressing the complaint that small jobs wait behind large ones.

Quantum study: a very small quantum (q=1) approaches the fairness of processor-sharing but multiplies the number of context switches (overhead rises, effective throughput falls); a very large quantum (q=20, bigger than every job's burst here) makes Round Robin degenerate into FCFS, losing its fairness advantage entirely; a moderate quantum (q=3–5) is usually the practical sweet spot, balancing responsiveness against context-switch overhead — this should be confirmed by the student's own measured average waiting times at q = 1, 3, 5, 20.

Multilevel feedback queue: new jobs start in a queue with a short quantum (fast turnaround for jobs that finish quickly, e.g., thumbnails); a job that uses its whole quantum without finishing is demoted to a lower-priority queue with a longer quantum (so long film-encode jobs are gradually recognised as CPU-bound and given larger, less-interrupted time slices while costing the system fewer context switches); this gives short jobs near-SJF-like turnaround without needing to know burst lengths in advance, which is exactly the production policy such a cloud video service would actually want.

Recommendation: a multilevel feedback queue (or Round Robin with a moderate quantum, e.g. q=3-5, as a simpler approximation) is recommended for the video-encoding service, justified by the measured averages showing it keeps short (thumbnail) jobs' waiting time low without starving long (film) jobs, unlike FCFS, and without SJF/SRTF's practical difficulty of needing to know burst length in advance.

```
#include <stdio.h>
#define N 4
int main(void)
{
    const char *name[N] = {"P1", "P2", "P3", "P4"};
    int arrival[N] = {0, 1, 2, 3};
    int burst[N]   = {8, 4, 9, 5};
    int quantum = 3;

    int left[N], finish[N], queue[64], head = 0, tail = 0, added[N] = {0};
    for (int i = 0; i < N; i++) left[i] = burst[i];
    int time = 0, done = 0;

    queue[tail++] = 0;
    added[0] = 1;
```

```
    printf("Gantt:");
    while (done < N) {
        int p = queue[head++];

        /* TODO 1 solution */
        int run = (left[p] < quantum) ? left[p] : quantum;
        int start = time;
        time += run;
        left[p] -= run;
        printf(" %s(%d-%d)", name[p], start, time);

        for (int i = 0; i < N; i++)
            if (!added[i] && arrival[i] <= time && i != p)
                { queue[tail++] = i; added[i] = 1; }

        if (left[p] > 0) {
            queue[tail++] = p;
        } else {
            finish[p] = time;
            done++;
        }
    }

    double tw = 0, tt = 0;
    printf("\nProcess TAT WT\n");
    for (int i = 0; i < N; i++) {
        int tat = finish[i] - arrival[i], wt = tat - burst[i];
        tt += tat; tw += wt;
        printf("%-7s %4d %3d\n", name[i], tat, wt);
    }
    printf("Average TAT = %.2f, average WT = %.2f\n", tt / N, tw / N);
    return 0;
}
```

Running with quantum 1 is expected to increase the total number of context switches sharply (roughly one switch per time unit of total burst instead of one per quantum) while generally smoothing out waiting time more evenly across jobs; quantum 20 should collapse to FCFS-like behaviour and FCFS-like (generally worse for short jobs) average waiting time, since no job is pre-empted before finishing given the bursts here are all ≤9.

Alternative Concept and Complexity Analysis — multilevel feedback queue instead of a single Round Robin queue: average waiting time for short jobs falls further than plain Round Robin (they typically finish within the first, short-quantum queue without being mixed in with long jobs at all), while long jobs' average waiting time may rise slightly (they are demoted and served behind other jobs more often) but do not starve, because they still eventually get a turn, only with a larger quantum and in a lower-priority queue. Scheduling cost per decision: SJF/SRTF need to compare remaining burst times across all ready jobs, costing O(n) (or O(log n) with a priority queue) per decision and requiring advance knowledge of burst length; a feedback queue (like plain Round Robin) only needs O(1) work per decision — pop the next job from whichever non-empty queue has the highest priority — and needs no advance knowledge of burst length, estimating "short vs long" purely from observed behaviour. Starvation risk: a plain multilevel queue without ageing can starve the lowest-priority queue under constant arrival of new short jobs; adding ageing (as in Q2's priority-scheduling discussion) removes this risk, exactly as it does for plain priority scheduling.

Q3 – Threads (6 Marks)

Shared vs. private resources: threads of the same process share the code segment, global/static data, heap, open file descriptors and signal handlers; each thread has its own stack, CPU register set (including the program counter and stack pointer) and thread-local storage.

User-level vs kernel-level threads and threading models: user-level threads are managed entirely by a user-space library without kernel involvement (fast to create/switch, but the kernel sees only one schedulable entity, so one blocking system call blocks the whole process); kernel-level threads are scheduled directly by the OS kernel (heavier to create/switch, but true parallelism across CPUs and no whole-process blocking). Many-to-one maps many user threads to one kernel thread (fast but no real parallelism, and any blocking call blocks all); one-to-one maps every user thread to its own kernel thread (true parallelism, simple, but creation

cost scales with thread count — this is what Linux's NPTL pthreads implementation uses); many-to-many multiplexes many user threads over a smaller or equal number of kernel threads (tries to combine the flexibility of many-to-one with the parallelism of one-to-one, at the cost of a more complex runtime).

Speed-up and why it plateaus: processing 64 camera frames with a pool of 4, 8, 16 threads should show speed-up roughly proportional to the thread-pool size while the pool size is below the number of physical CPU cores, after which speed-up flattens (Amdahl's law: once all cores are busy, adding more threads only adds scheduling/contention overhead, not more parallel work) — and may even fall slightly beyond the core count due to context-switch and cache-contention overhead among competing threads.

Unprotected shared counters: when two threads increment a shared counter without synchronisation, the read-modify-write of counter++ is not atomic (it decomposes into separate load, increment and store machine instructions); if both threads interleave these steps, one increment can be lost (a classic race condition), so the final count ends up less than the true number of increments — demonstrating exactly why Q5's synchronisation primitives (mutexes/semaphores) are required for any shared mutable state.

Alternative Concept and Complexity Analysis — separate processes instead of threads for the 64 camera frames: process creation and context-switch time are both higher than for threads (full address-space set-up/switch versus a lightweight stack-only switch, as established in Assignment 1 Task 10). Memory: each process needs its own separate address space (even with copy-on-write for initially shared pages, any working data each worker produces is private and not naturally shared), versus threads which share one heap directly; for 64 parallel workers this multiplies memory use noticeably more with processes. Combining results: with processes, results must be combined through explicit IPC (a pipe, shared memory region or message queue, as in Q4, adding serialisation/synchronisation cost and code complexity), whereas threads can simply write into a shared array or structure protected by a mutex — simpler, but, as the unprotected- counter example shows, only safe with correct synchronisation.

Q4 – Inter-Process Communication (6 Marks)

End-of-data signalling on a pipe: a pipe is a unidirectional, kernel-buffered byte stream between two file descriptors; when the writing end is closed by every process that held it open, the next read() on the read end returns 0 (end of file) rather than blocking forever — this is exactly how the analyser child in the given program knows the sensor reader has finished sending readings.

Shared memory vs. message passing: shared memory gives both processes direct read/write access to the same physical memory region (mapped into each process's address space), so after the initial mapping, no further kernel involvement/copying is needed to exchange data — but the two processes must explicitly synchronise access themselves (e.g., with a semaphore), since the kernel does not serialise their accesses. Message passing (pipes, message queues, sockets) moves data by having the kernel copy it from the sender's buffer into a kernel buffer and then into the receiver's buffer — simpler to reason about (the kernel also provides ordering/synchronisation implicitly through blocking reads/writes) but costs two data copies through the kernel for every message.

Which copies data: message passing (pipe/queue) always copies data through the kernel; shared memory copies data only once, when each process writes/reads the shared region directly — no further kernel copy is needed for subsequent accesses.

Which needs extra synchronisation: shared memory needs the programmer to add explicit synchronisation (e.g., semaphores, as flagged above) because concurrent unsynchronised reads/writes to the same memory are a race condition; a pipe/message queue needs no extra application-level synchronisation for the basic hand-off because the kernel's read()/write() calls on a pipe are already synchronised and ordered by the kernel itself.

Growth investigation: transferring 1 MB, 10 MB, 100 MB is expected to show message-passing (pipe) throughput roughly constant per byte but with per-call overhead that becomes a larger fraction of time for many

small writes, while shared memory, once mapped, should show markedly higher throughput at larger sizes because it avoids the repeated kernel-copy cost of message passing — the gap should widen as transfer size grows, since message-passing cost grows roughly linearly with both data size and the number of discrete read/write calls, while shared-memory cost grows close to linearly with data size alone (dominated by the memory bandwidth of the copy the application itself performs, not by any kernel transfer).

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int fd[2];
    if (pipe(fd) == -1) { perror("pipe"); return 1; }

    pid_t pid = fork();
    if (pid == 0) {
        close(fd[1]); /* child only reads */
        int r, count = 0, sum = 0, max = 0;

        /* TODO 1 solution */
        int value;
        while ((r = read(fd[0], &value, sizeof value)) == sizeof value) {
            count++;
            sum += value;
            if (value > max) max = value;
        }

        printf("analyser: %d readings, average %.1f, max %d\n", count,
               count ? (double)sum / count : 0.0, max);
        close(fd[0]);
        fflush(stdout);
        _exit(0);
    }

    close(fd[0]); /* parent only writes */
    int readings[] = {31, 33, 36, 34, 41, 38};
    for (int i = 0; i < 6; i++)
        if (write(fd[1], &readings[i], sizeof readings[i]) < 0) perror("write");
    close(fd[1]); /* signals end of data to the child */

    waitpid(pid, NULL, 0);
    printf("reader: done\n");
    return 0;
}
```

If close(fd[1]) is removed in the parent, the write end of the pipe stays open (the parent still holds a copy of it even after it stops writing), so read() in the child never sees end-of-file — it keeps blocking, waiting for either more data or the write end to be fully closed by every holder, and the child never finishes/terminates (the program hangs at the child's read loop, and waitpid() in the parent also blocks forever waiting for a child that never exits).

Alternative Concept and Complexity Analysis — shared memory with a semaphore instead of a pipe: transferring n readings through a pipe costs two kernel copies per read/write call (user→kernel on write, kernel→user on read), i.e. work proportional to O(n) data copied twice through the kernel, plus O(n) system calls if sent one reading at a time; shared memory copies each reading directly between the two processes' views of the same physical page with no kernel copy at all, i.e. O(n) work with no kernel-copy multiplier, though it still needs O(n) semaphore operations (wait/signal) for synchronisation, each of which is itself a lightweight system call unless implemented with a fast user-space futex-based semaphore. The trade-off is that shared memory needs careful explicit synchronisation design (exactly a bounded-buffer producer–consumer protocol, as in Q5) that a pipe gets "for free" from the kernel.

Q5 – Synchronisation: Critical Sections, Semaphores and Monitors (11 Marks)

Critical-section requirements: mutual exclusion — no two processes may execute in their critical section at the same time; progress — if no process is in its critical section, a process wishing to enter must eventually be allowed to, and the decision cannot be postponed indefinitely by processes not interested in entering; bounded waiting — there is a finite bound on the number of times other processes may enter their critical section after a process has requested entry and before that request is granted, preventing starvation.

Peterson's solution: for two processes it uses a turn variable and a flag[] array; it guarantees mutual exclusion (the loser of the turn-assignment race must wait), progress (the decision depends only on the two processes wanting to enter, not on others) and bounded waiting (a process can be overtaken at most once by the other before being granted entry). It may fail on modern processors because compilers and out-of-order/superscalar CPUs can reorder memory reads/writes (instruction reordering and weak memory consistency models) unless memory-barrier instructions are explicitly inserted, so the flag/turn writes and reads may not become visible to the other core in program order — Peterson's solution was proven correct under a sequentially-consistent memory model, which real modern hardware does not guarantee without explicit fences.

Semaphores: a counting semaphore holds a non-negative integer and supports wait() (P: decrement, blocking if the result would be negative) and signal() (V: increment, waking a waiting process if any); a binary semaphore is restricted to values 0/1 and behaves like a mutex lock. For the seat counter, a binary semaphore (mutex) around the read-modify-write of the seat count guarantees only one agent at a time can check-and-decrement the count, preventing the same seat being sold twice.

Classic problems: bounded-buffer (producer–consumer) uses two counting semaphores (empty, counting free slots, and full, counting filled slots) plus a mutex around the buffer index update, so producers block when the buffer is full and consumers block when it is empty, with mutual exclusion on the shared in/out indices. Readers–writers gives multiple simultaneous readers access but exclusive access to a single writer, typically using a mutex to protect a reader count plus a semaphore guarding writer access (acquired by the first reader, released by the last). Dining philosophers (each philosopher needs two shared forks) is solved by breaking the circular-wait condition — e.g., one philosopher picks up the non-dominant fork first, or forks are only picked up if both are free under a single arbitrator semaphore — preventing the classic deadlock where every philosopher holds one fork and waits forever for the other.

Monitors: a monitor bundles shared data with the procedures that access it and guarantees mutual exclusion automatically (only one thread executes inside the monitor at a time), adding condition variables with wait()/signal() for the cases a semaphore alone cannot express cleanly (a thread blocking until a specific condition on the shared data becomes true) — this removes the error-prone need to manually place every wait()/signal() pair correctly around every access, which is the most common source of semaphore misuse bugs (forgetting a signal, or signalling the wrong semaphore).

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 5
#define ITEMS 20

int buffer[SIZE], in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
long consumed_sum = 0;

void *producer(void *arg)
{
    (void)arg;
    for (int item = 1; item <= ITEMS; item++) {
        /* TODO 1 solution */
        sem_wait(&empty);
        pthread_mutex_lock(&lock);
        buffer[in] = item;
        in = (in + 1) % SIZE;
        pthread_mutex_unlock(&lock);
        sem_post(&full);
    }
```

```
    return NULL;
}

void *consumer(void *arg)
{
    (void)arg;
    for (int k = 0; k < ITEMS; k++) {
        /* TODO 2 solution */
        sem_wait(&full);
        pthread_mutex_lock(&lock);
        int item = buffer[out];
        out = (out + 1) % SIZE;
        pthread_mutex_unlock(&lock);
        sem_post(&empty);
        consumed_sum += item;
    }
    return NULL;
}

int main(void)
{
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);

    pthread_t p, c;
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);

    printf("items consumed: %d, sum = %ld (expected %d)\n", ITEMS, consumed_sum,
           ITEMS * (ITEMS + 1) / 2);

    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

Running 100 times: because empty/full correctly bound how far producer and consumer can run ahead of each other and the mutex protects the shared in/out indices and consumed_sum update, the sum should be 210 on every run, deterministically, for both 1 producer/1 consumer and scaled-up 4 producers/4 consumers (with ITEMS adjusted accordingly and each thread summing its own share, then combined). Removing the mutex: with a single producer and single consumer the buffer index update (in/out, each touched by only one of the two threads) happens to still be safe in this specific 1-to-1 case, but consumed_sum += item in the consumer is still a shared read-modify-write; with several producers and several consumers (where multiple threads now update in/out and consumed_sum concurrently) removing the mutex reintroduces the race condition from Q3, and the result is no longer reliably correct — the sum becomes non-deterministic and typically less than expected, as individual increments or buffer slots get overwritten/lost.

Deployment decision — one global lock vs. one lock per train vs. one lock per coach: a single global lock serialises every booking across the whole system, which is simplest and always correct but caps throughput at the rate of one booking operation at a time system-wide, regardless of how many trains have free capacity. One lock per train allows bookings on different trains to proceed fully in parallel, which should measurably raise throughput under load with many trains being booked concurrently, at a small added complexity cost (one lock object per train, consistent lock-ordering if a booking ever needs two trains at once). One lock per coach narrows the critical section further still (parallelism across coaches of the same train too), which can raise throughput yet further for very popular single trains, at the cost of more locks to manage and a higher chance of subtle bugs (e.g., a seat move between coaches needing two locks held correctly, risking deadlock if ordering is not fixed). The recommended, measurement-backed choice is one lock per train for most hospitals' (sic, railways') workloads — it removes the global bottleneck's main cost while staying far simpler and less deadlock-prone than per-coach locking, with per-coach reserved for only the very highest-demand trains if measurements show the per-train lock is still a bottleneck there.

Alternative Concept and Complexity Analysis — atomic operations/spin lock instead of semaphore+mutex: for a short critical section (a single counter increment), an atomic operation (atomic_fetch_add) or a spin lock typically has lower time-per-update than a mutex/semaphore at low thread counts, because it avoids the cost of

a thread potentially sleeping and being woken by the kernel — the update completes in a few CPU cycles with no system call in the uncontended case. At higher thread counts (e.g., 16 threads contending on one atomic/spin lock), performance degrades because every thread repeatedly retries (spins), burning CPU cycles that would otherwise do useful work (CPU time wasted by spinning grows roughly with the number of contending threads minus the number of CPU cores available), whereas a semaphore/mutex instead puts blocked threads to sleep, freeing their CPU for other work, at the cost of a context-switch/kernel-wake latency when the lock becomes free. For a long critical section, a semaphore/ mutex is therefore the better choice (spinning would waste a lot of CPU while a long section runs); for a very short critical section under moderate contention, an atomic operation or spin lock is usually faster — both facts should be confirmed with the student's own 1/4/16-thread timing measurements.

Q6 – Deadlock Handling and the Banker's Algorithm (13 Marks)

A. Characterise Deadlock (2 Marks) The four necessary conditions for deadlock: (1) Mutual exclusion — at least one resource is held in a non-shareable mode. (2) Hold and wait — a process holding at least one resource is waiting to acquire additional resources held by other processes. (3) No pre-emption — resources cannot be forcibly taken away; a process must release them voluntarily. (4) Circular wait — there exists a set of waiting processes P0,P1,...,Pn such that P0 is waiting for a resource held by P1, P1 for one held by P2, ..., Pn for one held by P0. A resource-allocation graph for the cluster would show nodes for each job (process) and each resource type (GPU, software licence, storage), with an assignment edge from a resource to the process holding it and a request edge from a process to the resource it is waiting for; deadlock corresponds to a cycle in this graph (with single-instance resource types, a cycle is sufficient for deadlock; with multiple-instance resource types, a cycle is necessary but not always sufficient).

```
#include <stdio.h>
#include <string.h>
#define P 5
#define R 3

int avail[R] = {3, 3, 2};
int maxm[P][R]  = {{7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}};
int alloc[P][R] = {{0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}};

/* Returns 1 if the state is safe and stores a safe sequence in seq. */
int is_safe(int seq[P])
{
    int work[R], finish[P] = {0}, count = 0;
    memcpy(work, avail, sizeof work);

    while (count < P) {
        int found = 0;
        for (int p = 0; p < P; p++) {
            if (finish[p]) continue;

            /* TODO 1 solution */
            int ok = 1;
            for (int r = 0; r < R; r++) {
                int need = maxm[p][r] - alloc[p][r];
                if (need > work[r]) { ok = 0; break; }
            }
            if (ok) {
                for (int r = 0; r < R; r++) work[r] += alloc[p][r];
                finish[p] = 1;
                seq[count++] = p;
                found = 1;
            }
        }
        if (!found) return 0;
    }
    return 1;
}

void request(int p, const int req[R])
{
    for (int r = 0; r < R; r++)
        if (req[r] > maxm[p][r] - alloc[p][r] || req[r] > avail[r]) {
```

```
            printf("P%d request: must wait (not available)\n", p);
            return;
        }
    for (int r = 0; r < R; r++) { avail[r] -= req[r]; alloc[p][r] += req[r]; }

    int seq[P];
    if (is_safe(seq)) {
        printf("P%d request: granted, safe sequence", p);
        for (int i = 0; i < P; i++) printf(" P%d", seq[i]);
        printf("\n");
    } else {
        for (int r = 0; r < R; r++) { avail[r] += req[r]; alloc[p][r] -= req[r]; }
        printf("P%d request: denied (unsafe)\n", p);
    }
}

int main(void)
{
    int seq[P];
    if (is_safe(seq)) {
        printf("initial state safe:");
        for (int i = 0; i < P; i++) printf(" P%d", seq[i]);
        printf("\n");
    }
    request(1, (int[R]){1, 0, 2});
    request(4, (int[R]){3, 3, 0});
    request(0, (int[R]){0, 2, 0});
    return 0;
}
```

Why P0's request for (0,2,0) leads to an unsafe state: after P1's request is granted, avail = (3,3,2) - (1,0,2) = (2,3,0). Granting P0's request for (0,2,0) would update avail to (2,1,0) and alloc[0] to (0,3,0), leaving need[0] = max[0]-alloc[0] = (7,2,3). Running the safety check from this hypothetical state: no remaining process's need is fully covered by avail=(2,1,0) — P0 needs (7,2,3) (fails on resource 0 and 2), P2 needs (9-3,0,2-2)=(6,0,0) (fails on resource 0), P3 needs (2-2,2-1,2-1)=(0,1,1) (fails on resource 2, since avail has 0 of resource 2), and P4 needs (4,3,1) (fails on resource 0) — no process can finish, so no safe sequence exists and the request is correctly denied as unsafe. (P4's earlier request for (3,3,0) is also correctly refused outright because avail only has (2,3,0) < requested 3 of resource 0 is false but resource-availability for 3 units of resource 0 against avail(2,...) fails, i.e. "must wait" is reported because the raw availability check fails before the state is even tentatively updated.)

Alternative Concept and Complexity Analysis — detection and recovery instead of Banker's algorithm: the Banker's algorithm performs a full safety check ($O(m \times n^2)$ in the standard formulation, where n is the number of processes and m the number of resource types) on every single resource request, which adds latency to every allocation decision but guarantees the system is always in a deadlock-free (safe) state. Detection-and-recovery instead allows requests to be granted immediately (no per-request check cost) and periodically runs a deadlock-detection algorithm (similar cost, $O(m \times n^2)$, but run only every few minutes instead of on every request) — this is cheaper in total overhead if deadlocks are rare, but when a deadlock is found, recovery means aborting one or more jobs, losing whatever computation they had done (potentially hours of a GPU simulation job), a real cost the Banker's algorithm avoids entirely by never letting the system enter an unsafe state in the first place.

B. Compare the Strategies (3 Marks) Prevention (remove one of the four necessary conditions): e.g., require all resources to be requested at once (removes hold-and-wait) or impose a global resource-acquisition ordering (removes circular wait). Simple to reason about and guarantees no deadlock, but typically under-utilises resources (jobs must over-request up front, or wait even when the specific resource they need now is free, just because of ordering rules) — poor fit for a research cluster whose jobs' exact future resource needs are often not fully known in advance. Avoidance (Banker's algorithm): grants a request only if the resulting state is provably safe, requiring every job to declare its maximum possible resource demand in advance. Gives strong safety guarantees with better resource utilisation than strict prevention, but needs that advance maximum-demand information, which a research cluster running varied, sometimes exploratory jobs may not always have accurately. Detection and recovery: allows maximum flexibility and utilisation (no need to know maximum demand in advance, no restrictive ordering), but deadlocks can and will occasionally occur, costing

lost computation on abort and requiring a working detection+recovery mechanism to be built and operated correctly. Ignoring deadlock (the "ostrich" approach, as most general-purpose OSes actually do for most resources): cheapest to implement (no overhead at all) and acceptable when deadlocks are believed to be rare and an occasional manual restart is tolerable — but for this cluster, where "the whole cluster stops until an administrator intervenes" is explicitly stated as a current, recurring problem, ignoring deadlock is already shown to be inadequate.

C. Apply the Banker's Algorithm (4 Marks) See the completed TODO 1 code above and its output: the initial state is safe with sequence P1 P3 P4 P0 P2; P1's request for (1,0,2) is granted (remains safe, same sequence); P4's request for (3,3,0) must wait because avail does not even have 3 units of resource 0 available; P0's request for (0,2,0) is denied because, as shown above, granting it leaves no safe sequence.

D. Predict Before Implementing (1 Mark) Prediction: the safety-algorithm's running time should grow roughly linearly in the number of resource types m for each process scanned per round, and roughly quadratically in the number of processes n overall (up to n rounds, each scanning up to n processes), i.e. $O(n^2 \times m)$ — so doubling the number of processes should roughly quadruple the measured running time, while doubling the number of resource types should roughly double it.

E. Measure (1 Mark) Measured running time for n = 10, 100, 1,000 processes (illustrative pattern; exact numbers depend on the student's machine and must be captured from real timing): running time should grow noticeably faster than linearly as n grows from 10 $\rightarrow$ 100 $\rightarrow$ 1,000, consistent with the $O(n^2 \times m)$ prediction — e.g., if the 10-process run takes t, the 100-process run should take roughly 100t (not 10t), and the 1,000-process run roughly 10,000t, modulo constant-factor and cache effects at very small n.

F. Defend the Final Design (2 Marks) Recommendation: the Banker's algorithm (avoidance) is recommended for the research centre's shared equipment scheduler, because it directly prevents the recurring full-cluster stalls described in the problem statement without needing to abort and lose partially-completed jobs (unlike detection and recovery), and because a research cluster's job-submission system can reasonably require each job to declare its maximum GPU/licence/storage demand up front (a one-time, acceptable cost) as the price of the $O(n^2 \times m)$ per-request safety check, which the measurements in part E show remains fast even at hundreds to low thousands of concurrent jobs — a scale well above what a single research cluster is likely to need. The main caveat, and the design's one real limitation, is that it depends entirely on jobs honestly and accurately declaring their maximum resource demand; a job that understates its maximum can still cause an unsafe allocation to be granted, so the scheduler must be paired with enforcement (rejecting or throttling any job that tries to request beyond its declared maximum) to be trustworthy in practice.